

AR Industrial Safety Training Platform

Repository-based seven-day workflow for a four-person prototype team

7	4	2 + 1
DAYS	TEAM MEMBERS	CORE + BONUS AR MODULES

Project overview

The repository currently contains an empty Unity project skeleton, an initial Node package, a Vite/React starter nested under the backend, and placeholder folders for assets, documentation and localisation. The team will correct the repository layout on Day 1, prove one complete training journey early, reuse that foundation for the second module, and reserve the final two days for testing and demonstration readiness.

Required project features

- Android AR app with Fire and Explosion Response: exit identification, extinguisher use and evacuation sequencing overlaid on real surroundings via the phone camera.
- Gas Leak and Confined Space Protocol: hazard-zone recognition, PPE selection and buddy-system procedures simulated in AR.
- Optional Jharkhand mine-worker-specific bonus module, attempted only after the required scope is stable.
- Assessment engine; QR-based certificate generation and verification; Hindi and Santali localisation; offline functionality; low-poly models for low- and mid-range devices; and an admin web dashboard.

Certificate wording: Use **DGMS-aligned competency-based certificate**. Do not describe the prototype credential as official DGMS certification or imply DGMS issuance, approval or accreditation.

Repository setup and feature ownership

Day 1 begins with repository correction. Each feature then has one accountable lead and named integration support.

PERSON	ROLE	ACCOUNTABILITY
Person 1	Unity AR and training modules	Android AR app, training interactions, local assessment events and final APK
Person 2	Backend and data	Node.js, Express, PostgreSQL, scoring, sync APIs and QR certificates
Person 3	Admin dashboard	React dashboard, worker results, analytics and public QR verification
Person 4	Offline, language and assets	Offline queue, Hindi and Santali, low-poly assets and test coordination

Feature-to-owner mapping

FEATURE	LEAD	SUPPORT
Android AR app and two required modules	Person 1	Person 4 assets; Person 2 results
Jharkhand mine-worker bonus module	Person 1	Person 4 content and assets
Assessment engine	Person 2	Person 1 event capture
QR generation and verification	Person 2	Person 3 verification page
Hindi and Santali localisation	Person 4	Persons 1 and 3 integration
Offline functionality	Person 4	Person 2 sync API; Person 1 app
Low-poly device support	Person 4	Person 1 scene profiling
Admin web dashboard	Person 3	Person 2 APIs

Repository actions before feature work: Move **backend/admin-dashboard** to root-level **admin-dashboard**; keep the backend and dashboard as separate applications; create the real Unity project through Unity Hub; add a project `.gitignore`; exclude tmp render files; decide whether output PDFs belong in Git; and commit the clean baseline. Person 2 owns the shared data contract for worker, module, attempt and certificate identifiers, assessment events, pass criteria, API responses and sync states.

PERSON 1

Unity AR and training modules

Own the Android training experience and deliver the final integrated APK.

DAY AND FOCUS	WORK AND DELIVERABLE
Day 1 - Foundation	Create the Unity project with AR Foundation and ARCore. Configure Android builds, camera permission and plane detection. Define both scenario sequences and assessment events with Person 2. Deliverable: Installable AR starter build and interaction checklist.
Day 2 - Fire prototype	Build Fire and Explosion Response with exit identification, extinguisher selection/use and evacuation sequencing in the phone-camera view. Use stable language keys. Deliverable: Playable Fire scenario covering all three actions.
Day 3 - Fire assessment	Record correct and incorrect actions, action order and completion time. Display a score, save it locally and submit a sample result to the backend. Deliverable: Assessed Fire attempt stored locally and accepted by the API.
Day 4 - Gas prototype	Build Gas Leak and Confined Space Protocol with hazard-zone recognition, PPE selection and buddy-system procedures. Reuse the established interaction framework. Deliverable: Playable Gas scenario covering all required procedures.
Day 5 - Required modules complete	Finish Gas assessment, connect both modules to the result flow, integrate Hindi and Santali, and check retries, scene resets and consistent attempt identifiers. Deliverable: Two complete modules with assessment and language selection.
Day 6 - Integration	Run both modules offline, restart the app, reconnect and confirm syncing. Fix tracking, scoring and scene issues. Attempt bonus work only after the shared acceptance gate passes. Deliverable: Release-candidate APK with both required modules.
Day 7 - Delivery	Fix release blockers, build the final APK and rehearse training, assessment and certificate status. Record a backup walkthrough and document the Unity build process. Deliverable: Final APK, recording and Unity handover notes.

PERSON 2

Backend, database and certificate system

Own persistent records, result validation, synchronisation and certificate issuance.

DAY AND FOCUS	WORK AND DELIVERABLE
Day 1 - Data contract	Create Node.js, Express and PostgreSQL foundations. Define workers, modules, attempts and certificates, roles, attempt IDs, scoring version, sync responses and prototype pass criteria. Deliverable: Schema, endpoint contract and sample API data.
Day 2 - Core API	Implement authentication plus worker, module and attempt endpoints. Seed demonstration workers and the two required modules. Restrict management routes to administrators. Deliverable: Usable core API and demonstration credentials.
Day 3 - Assessment engine	Implement versioned scoring, pass/fail rules and result validation. Preserve completed attempts and use attempt IDs to make retries duplicate-safe. Deliverable: Assessment storage proven with pass, fail and duplicate cases.
Day 4 - QR certificates	Issue a certificate only for a validated passing result. Store a unique certificate ID and expose a QR verification URL with only necessary public fields. Deliverable: Working certificate generation and verification APIs.
Day 5 - Sync and analytics	Finish queued-result syncing and dashboard summary endpoints. Return an acknowledgement per attempt and leave unacknowledged results pending on the client. Deliverable: Stable sync API, metrics and eligibility checks.
Day 6 - Failure testing	Test expired login, malformed results, network interruptions, duplicate requests and invalid certificate IDs. Reconcile app attempts, database rows and dashboard totals. Deliverable: Integration defects resolved and records consistent.
Day 7 - Backend release	Freeze the API, prepare demonstration data, verify backend reachability and QR URLs, and save schema, configuration instructions and a restorable backup. Deliverable: Running backend, database backup and API handover.

PERSON 3

React admin dashboard

Own the administrator experience and browser-based certificate verification journey.

DAY AND FOCUS	WORK AND DELIVERABLE
Day 1 - Dashboard correction	Move the existing Vite/React starter from backend/admin-dashboard to root-level admin-dashboard. Remove the starter screen, then add routing, navigation and page shells for overview, workers, results and certificates. Deliverable: Separated dashboard application and endpoint checklist.
Day 2 - Workers	Build login, summary cards, worker listing and worker details using contract-shaped sample data until live endpoints are ready. Include loading, empty and error states. Deliverable: Login and worker list with clear data states.
Day 3 - Results	Display module, score, pass/fail, completion time and worker history. Add worker and module filters and align all fields with the assessment response. Deliverable: Results and worker-detail pages ready for real attempts.
Day 4 - Certificates	Build certificate management and a public verification page opened by the QR URL. Show valid, unknown and invalid/revoked states with DGMS-aligned wording. Deliverable: Sample QR opens the matching verification record.
Day 5 - Live APIs	Replace sample data with live workers, results, certificates and summary APIs. Add essential charts and filters and integrate agreed translated labels. Deliverable: Live dashboard showing results from both modules.
Day 6 - Journey testing	Verify offline-synced results appear once. Test administrator access, invalid QR links, empty lists, server failures and translated text wrapping. Deliverable: Admin and public verification flows pass integration checks.
Day 7 - Dashboard delivery	Prepare clean demonstration data and rehearse worker lookup, result review and QR verification. Freeze the UI and document setup, sign-in and page use. Deliverable: Demo-ready dashboard and administrator guide.

PERSON 4

Offline, localisation, assets and testing

Own the cross-cutting mobile support work and coordinate device-level quality checks.

DAY AND FOCUS	WORK AND DELIVERABLE
Day 1 - Baseline	Add the shared .gitignore and help verify the clean repository baseline. Choose low-poly assets and scene budgets, define language keys, confirm the Santali script/reviewer, design sync states and list target phones. Deliverable: Clean baseline, asset list, translation sheet and device list.
Day 2 - Fire support	Optimise exit, fire and extinguisher assets. Prepare Hindi and Santali strings for shared UI and Fire instructions. Test fonts and glyph rendering in Unity. Deliverable: Import-ready Fire assets and tracked language strings.
Day 3 - Offline queue	Implement SQLite or local storage for completed attempts and queued uploads. Persist attempt IDs and scoring version and restore pending items after restart. Deliverable: A Fire attempt survives loss of connectivity and restart.
Day 4 - Gas and sync	Deliver hazard-zone and PPE assets plus Gas and buddy-system strings. Connect the queue to the sync API and remove entries only after acknowledgement. Deliverable: Gas assets integrated; results sync without duplicates.
Day 5 - Language and performance	Complete Hindi and Santali review, bundle required offline content, profile both scenes and reduce model, texture, lighting and memory costs. Deliverable: Localised offline build with recorded device observations.
Day 6 - System testing	Lead airplane-mode, restart, reconnect, language-switching and device tests. Verify each queued attempt reaches the dashboard exactly once and coordinate retesting. Deliverable: Signed-off checklist or explicit release-blocker list.
Day 7 - Final quality check	Install the final APK on demonstration phones, preload content, run the full journey and prepare asset licences, test evidence and known limitations. Deliverable: Verified demo devices and final test report.

Team checkpoints and acceptance

Hold a 15-minute morning dependency check and an end-of-day demonstration from the shared build. Give every issue one owner and merge working increments daily.

CHECKPOINT	FOCUS	EVIDENCE REQUIRED
Day 1	Scope and interfaces	AR runs on a phone; scenario flows, API contract, scoring rules, language keys and ownership are agreed.
Day 2	First interactions	Fire is playable; core endpoints and dashboard skeleton exist; initial assets and translations are delivered.
Day 3	First complete data flow	A Fire attempt produces a score, survives local storage and is accepted by the backend.
Day 4	Second module and QR	Gas is playable; a sample certificate QR resolves; sync client and backend exchange acknowledgements.
Day 5	Required-feature freeze	Both modules, assessment, QR, languages, offline queue, low-poly assets and dashboard are integrated.
Day 6	Acceptance and bonus gate	Core journeys and failure cases pass on available phones, with no unresolved demonstration blocker.
Day 7	Release rehearsal	APK, backend and dashboard reproduce the end-to-end journey; handover and backup recording are ready.

Final demonstration sequence

Select Hindi or Santali, complete both required AR modules and view each assessment. Complete one attempt offline, close and reopen the app, reconnect, and confirm the result appears exactly once in the dashboard. For a server-validated pass, generate a certificate and scan its QR to open the matching verification record.

Offline boundary

Preloaded training content, instructions, assessment and saved results work without connectivity. New certificate issuance and live QR verification require the backend. Show a pending state until results sync and are validated; do not present cached status as live verification.

Priority order and completion criteria

Localisation, offline storage and device constraints begin on Day 1 because they shape the core design.

1	Correct and prove the foundation Separate the dashboard from the backend, commit a clean baseline, confirm AR support and establish working mobile, backend and dashboard projects.
2	Finish the Fire journey Complete Fire from interaction through assessment, local persistence and backend storage.
3	Complete Gas and offline sync Reuse the working interaction framework and prove restart-safe, duplicate-safe synchronisation.
4	Complete the required experience Finish QR certificates, Hindi and Santali, low-poly optimisation and live admin views.
5	Stabilise and rehearse Use Day 6 for integration and Day 7 for release, documentation and demonstration practice.
6	Consider the bonus Attempt a small Jharkhand module only after every required feature passes the Day 6 gate.

Definition of done

Both required modules cover every requested procedure and produce repeatable assessment results. Hindi and Santali text is reviewed and readable. Preloaded training and results persist offline, reconnection creates no duplicate attempts, and the dashboard matches backend records. Only a validated passing attempt receives a QR certificate, using DGMS-aligned wording and a working verification page.

Device support and handover

Record the Android models tested and their observed limitations. Hand over the APK, source projects, database schema, API and setup instructions, language files, asset licences, test evidence and backup demonstration recording.